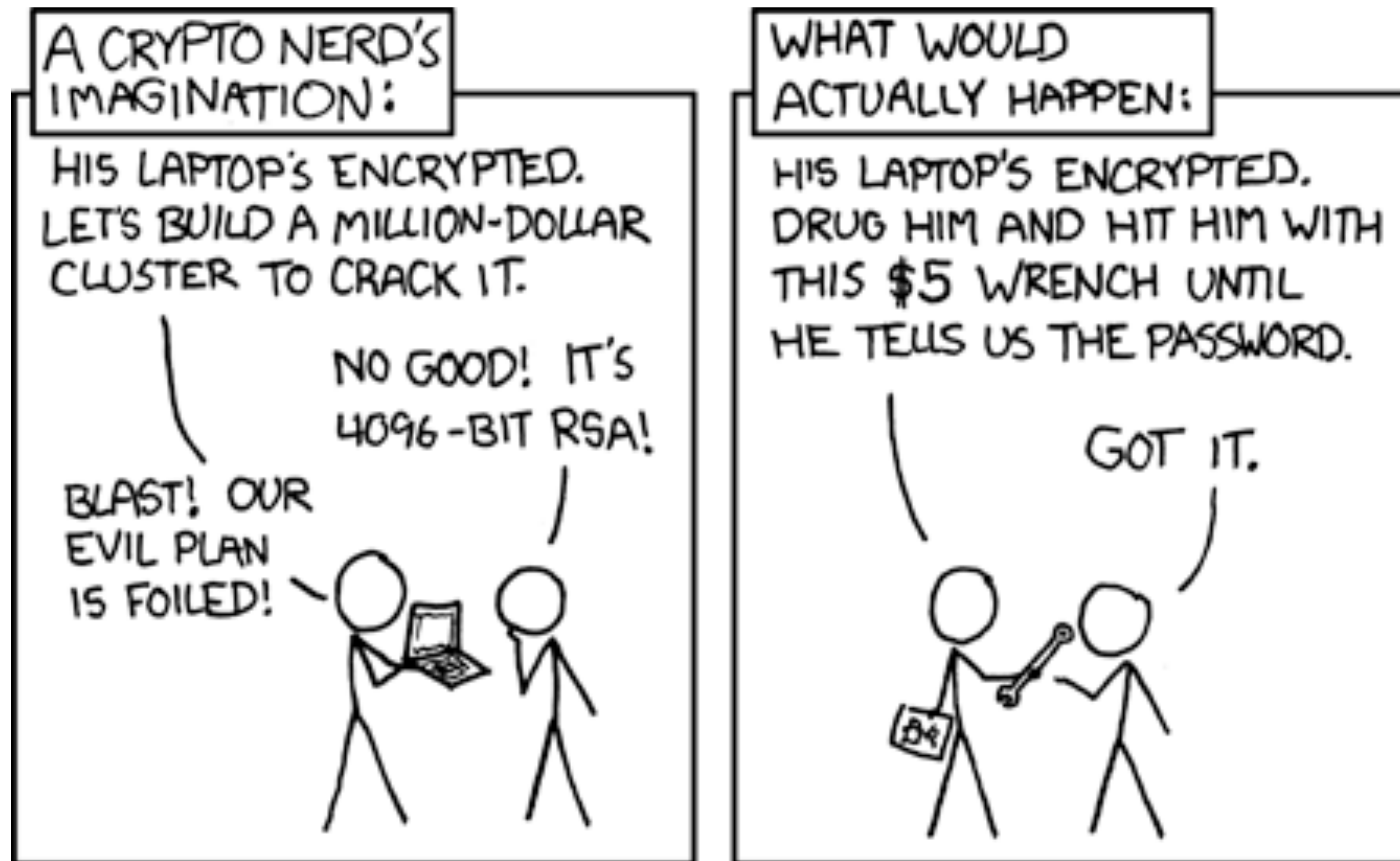


Briefest of Primers on Cryptography



Randall Munroe
<https://xkcd.com/538/>

Learning Goals

- ▶ Brief Primer on Cryptography
 - ▶ Symmetric encryption
 - ▶ Hash functions
 - ▶ Message Authentication Codes
 - ▶ Asymmetric encryption
 - ▶ Hybrid encryption
 - ▶ Digital Signatures
 - ▶ Pitfalls when implementing crypto

Learning Goal F3CE7B

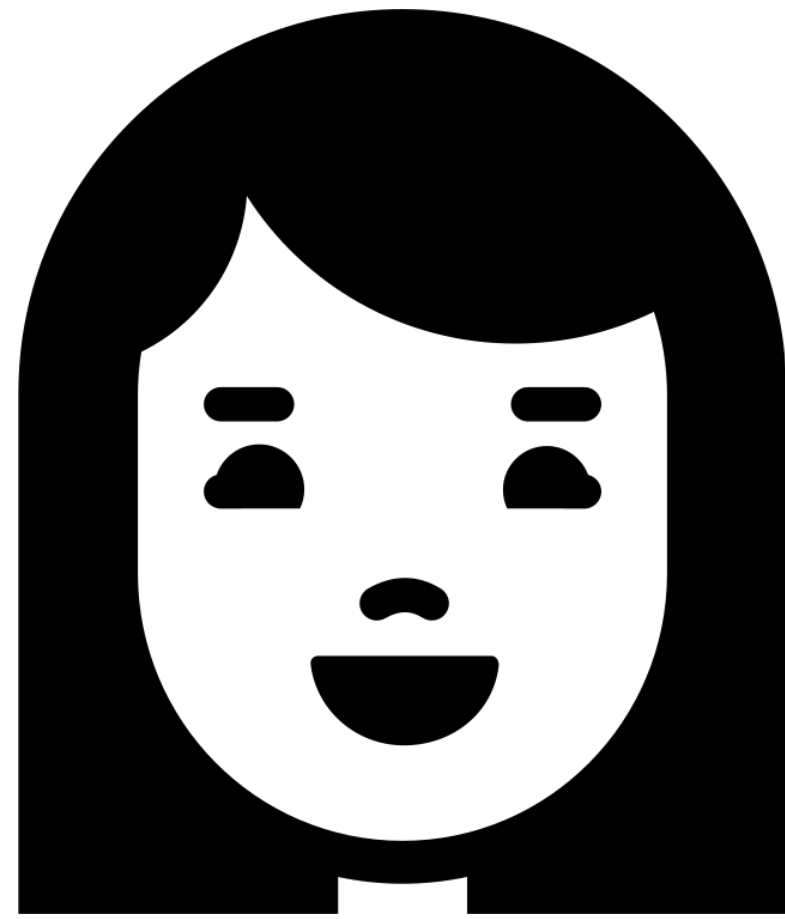
What is Cryptography?

Cryptography Definition by NIST

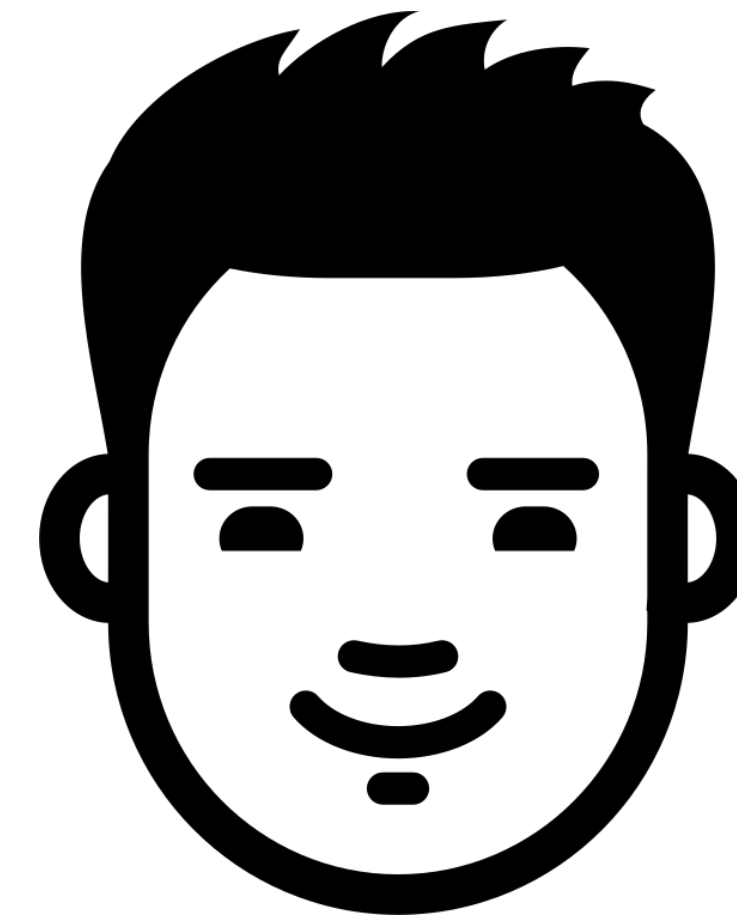
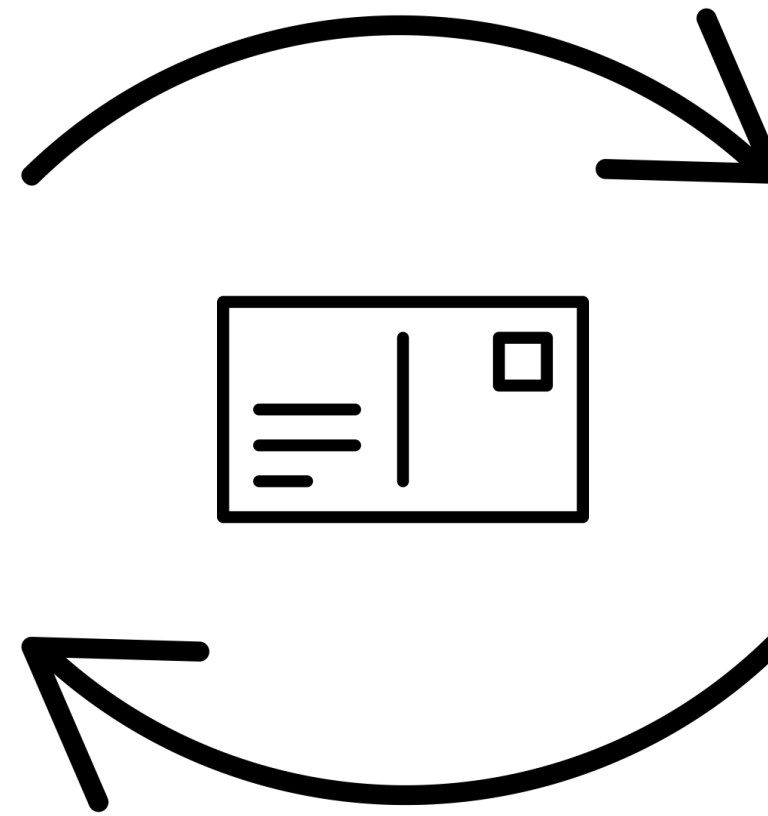
The discipline that embodies the principles, means, and methods for providing information security, including confidentiality, data integrity, non-repudiation, and authenticity.

- CNSSI 4009-2015

The basic setup

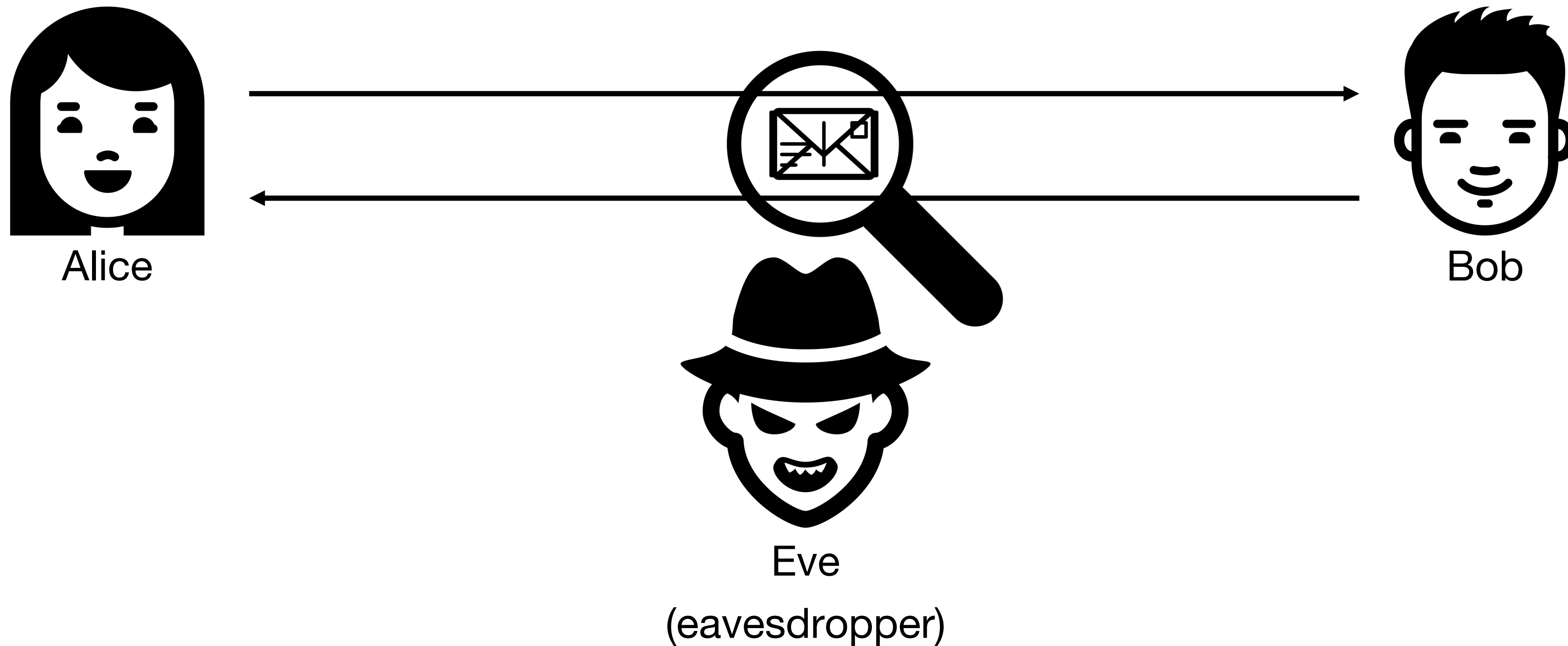


Alice

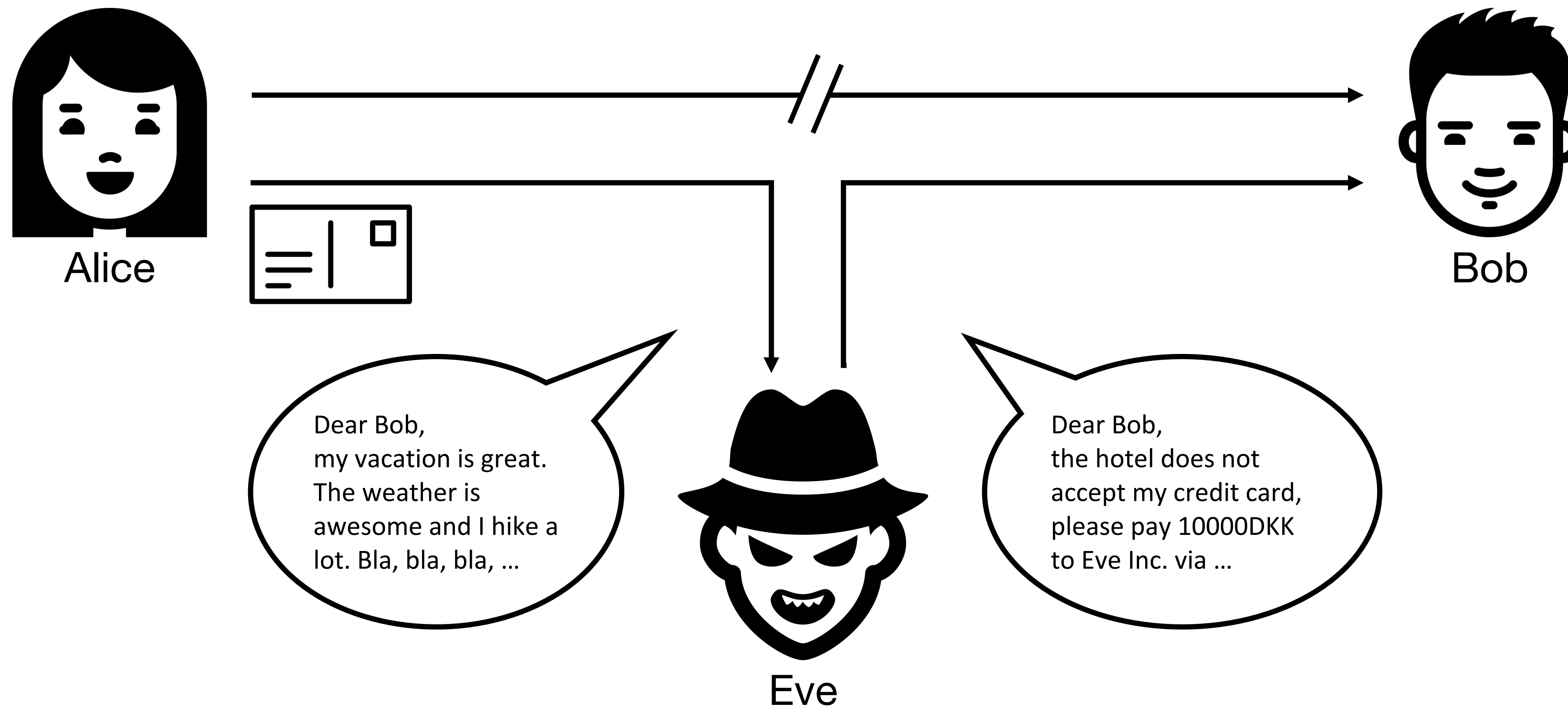


Bob

The basic setup – Confidentiality



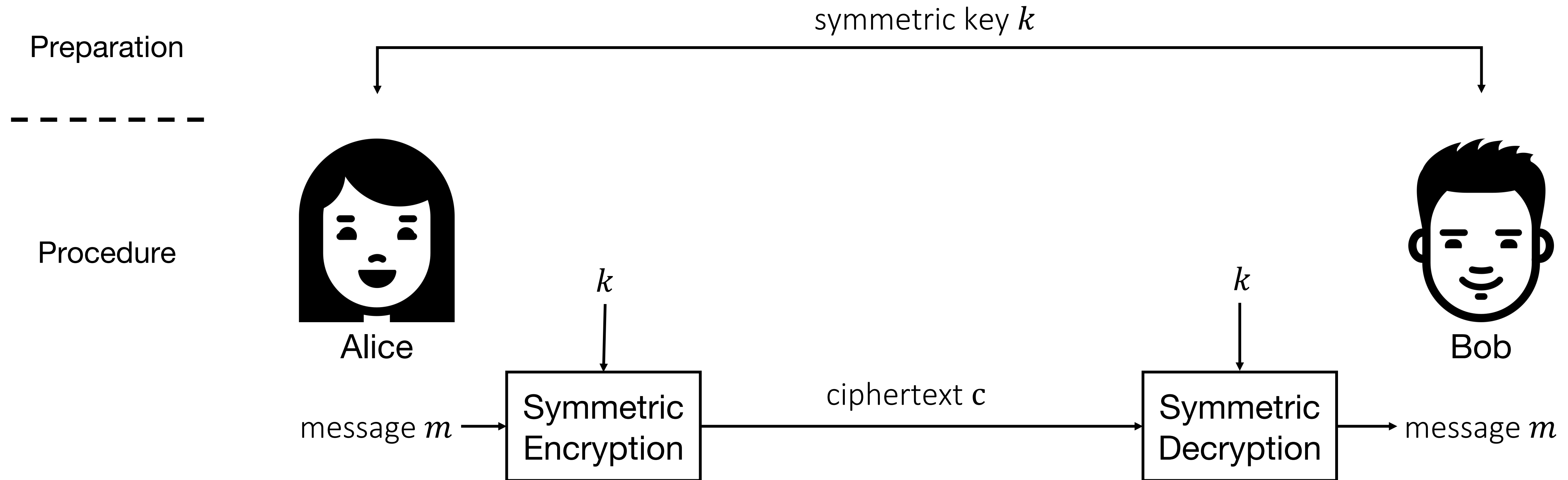
The basic setup – Integrity & Authenticity



Cryptographic Primitives (we will be looking at)

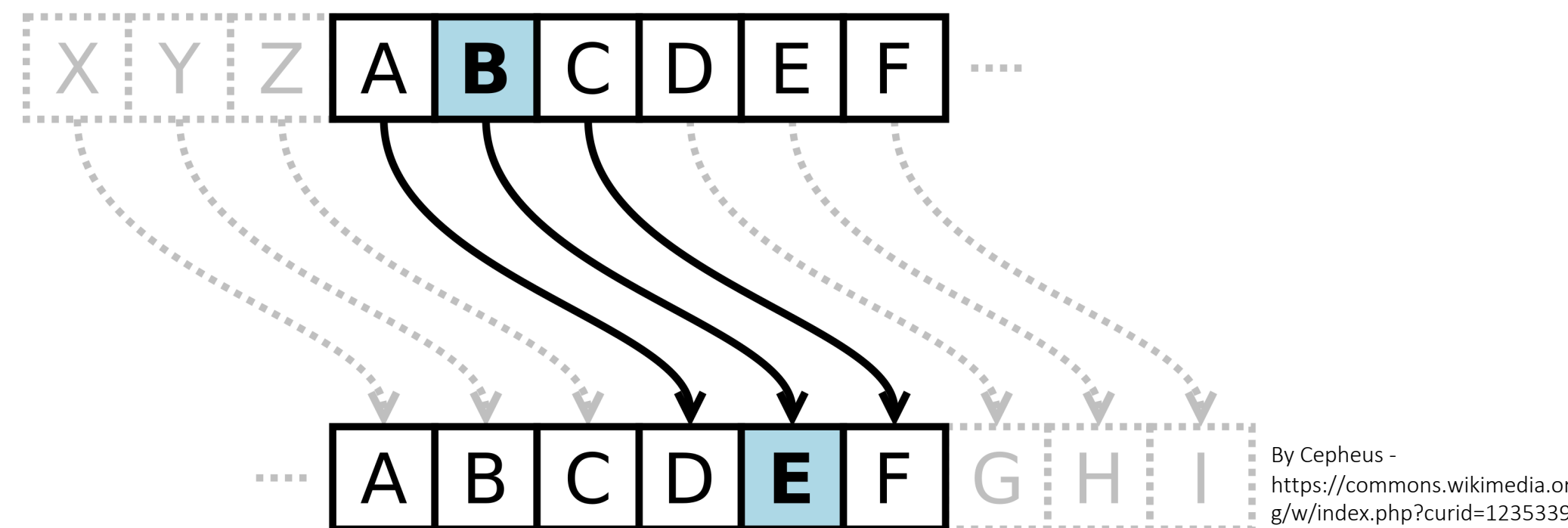
- ▶ Symmetric cryptography → Only one key for everything
 - ▶ Symmetric Encryption (Confidentiality)
 - ▶ Hash Functions (Integrity)
 - ▶ Message Authentication Codes (Authenticity)
- ▶ Asymmetric cryptography → Different keys for different operations
 - ▶ Asymmetric Encryption (Confidentiality)
 - ▶ Digital Signatures (Integrity & Authenticity)

Symmetric Encryption – The basic principle



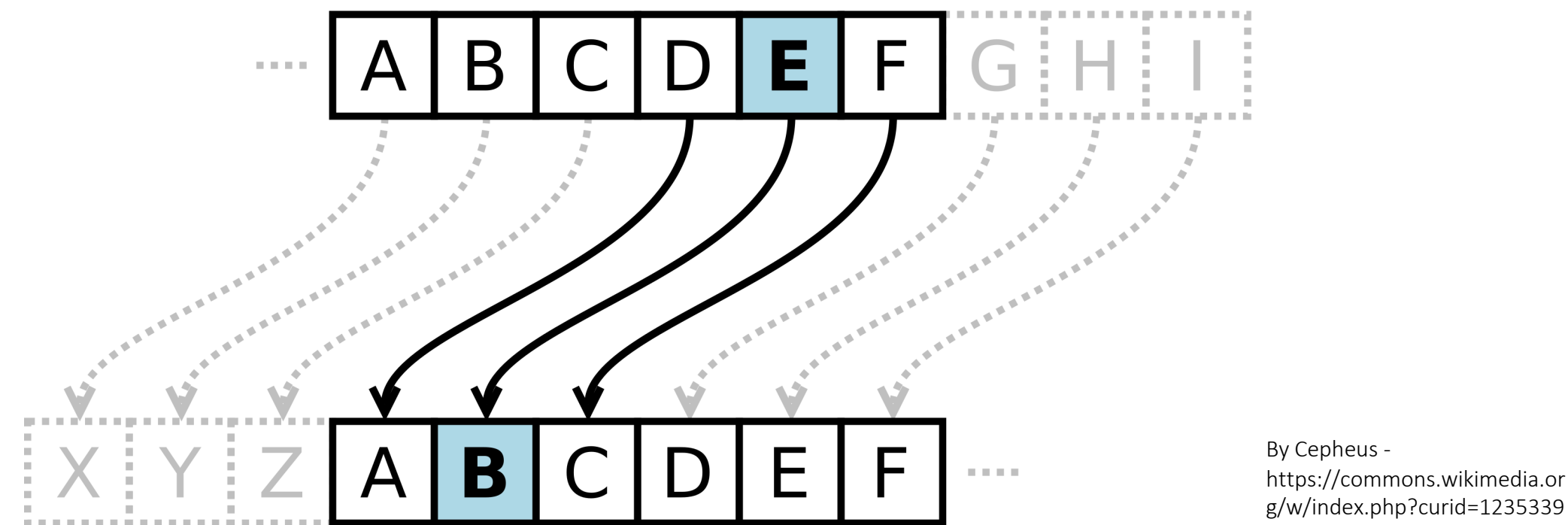
Symmetric Encryption – Caesar Cipher

- ▶ Substitution cipher, i.e., every character is replaced by another
- ▶ The key k is an integer and determines the substitution character
- ▶ Encryption: Alphabet is “rotated” forward by k steps, e.g., 3



Symmetric Encryption – Caesar Cipher

- Decryption: Alphabet is “rotated” backwards by k steps, e.g., 3



- Special case with $k = 13$ was used by Julius Caesar
- Try to encrypt the word “HEJ” with $k = 3$
- Is this cipher secure? If so, why? If not, why not?

Interlude: Kerckhoff's Principle(s)

- ▶ Kerckhoff was a French academic in 19th century
- ▶ Published in 1883 “La Cryptographie Militaire”
- ▶ Therein he stated six design rules, the most famous one:

*[The system] should not require secrecy,
and it should not be a problem if it falls into enemy hands.*

- ▶ The meaning is: The key should be the only secret, not the mechanism
- ▶ Secrecy based on the mechanism is called Security by Obscurity
- ▶ **Security by Obscurity is bad** as we have seen with the Caesar cipher

Symmetric Encryption – One-Time Pad

- ▶ Message m and key k are added using modular addition

Example: $m = 5, k = 15, \text{modulus } 17$

$$m + k \equiv 5 + 15 \equiv 20 \equiv 3 \text{ mod } 17$$

- ▶ In practice bit-wise XOR, modular addition with *modulus* 2

Example: $m = 1001, k = 1010,$

$$m \otimes k \equiv 1001 \otimes 1010 \equiv 0011$$

- ▶ In General: Addition for encryption, subtraction for decryption
- ▶ For XOR: Encryption and decryption are same operation
- ▶ Try to encrypt the message "01101101" with $k = 10100100$

Symmetric Encryption – One-Time Pad

► A more complex example

A	B	C	D	E	F	G	H	I	J	K	L	M	N
00000	00001	00010	00011	00100	00101	00110	00111	01000	01001	01010	01011	01100	01101

O	P	Q	R	S	T	U	V	W	X	Y	Z	.	_
00000	00001	00010	00011	00100	00101	00110	00111	01000	01001	01010	01011	01100	01101

► Decrypt the ciphertext $c = 11101\ 11100\ 11110$

► with the key $k = 11010\ 11000\ 10111 = 00111\ 00100\ 01001 = \text{HEJ}$

► and then with $k = 11100\ 10110\ 11010 = 00001\ 01010\ 00100 = \text{BYE}$

► Can you you guess the right key?

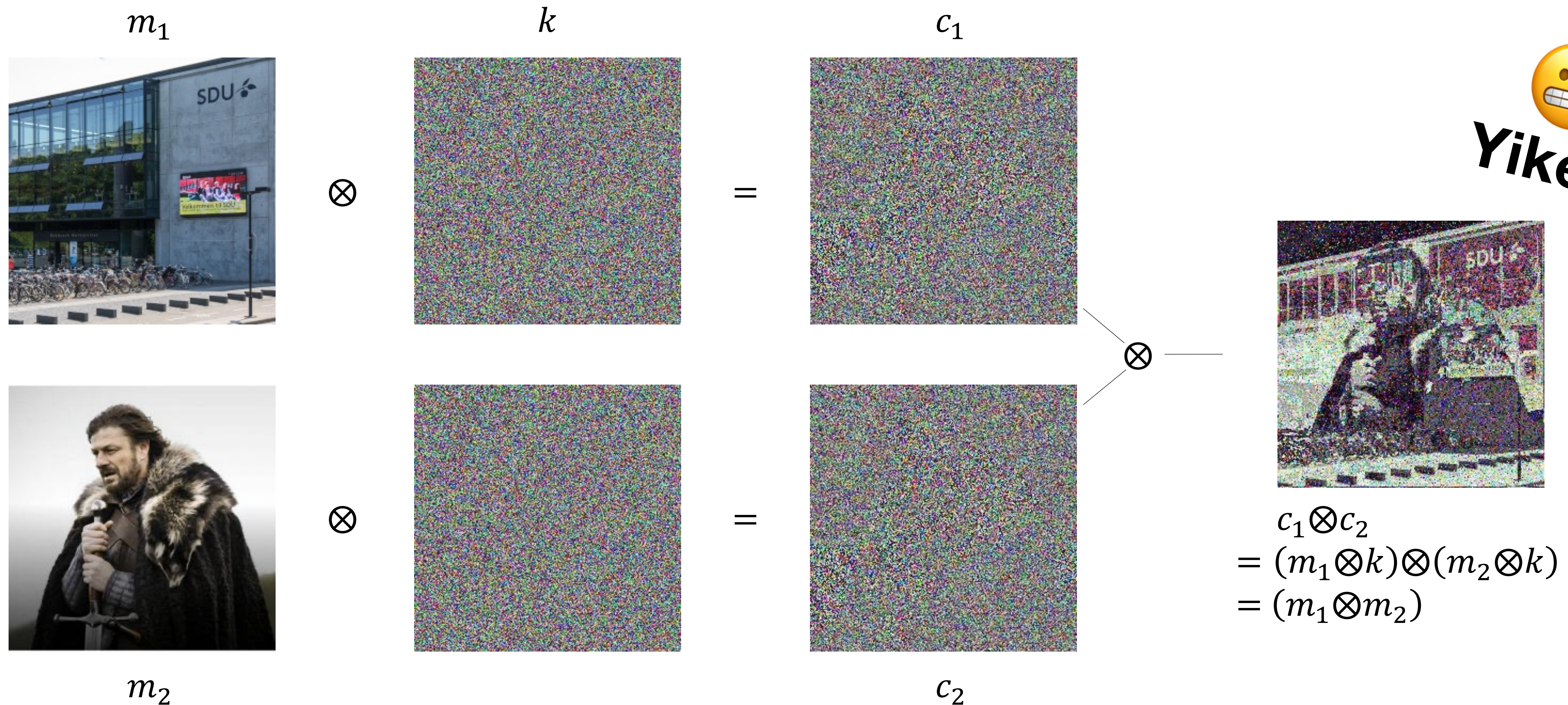
► Information-theoretically secure $\rightarrow$ Cannot be cracked

► But: Never re-use a key!

Is this practical?

Symmetric Encryption – One-Time Pad

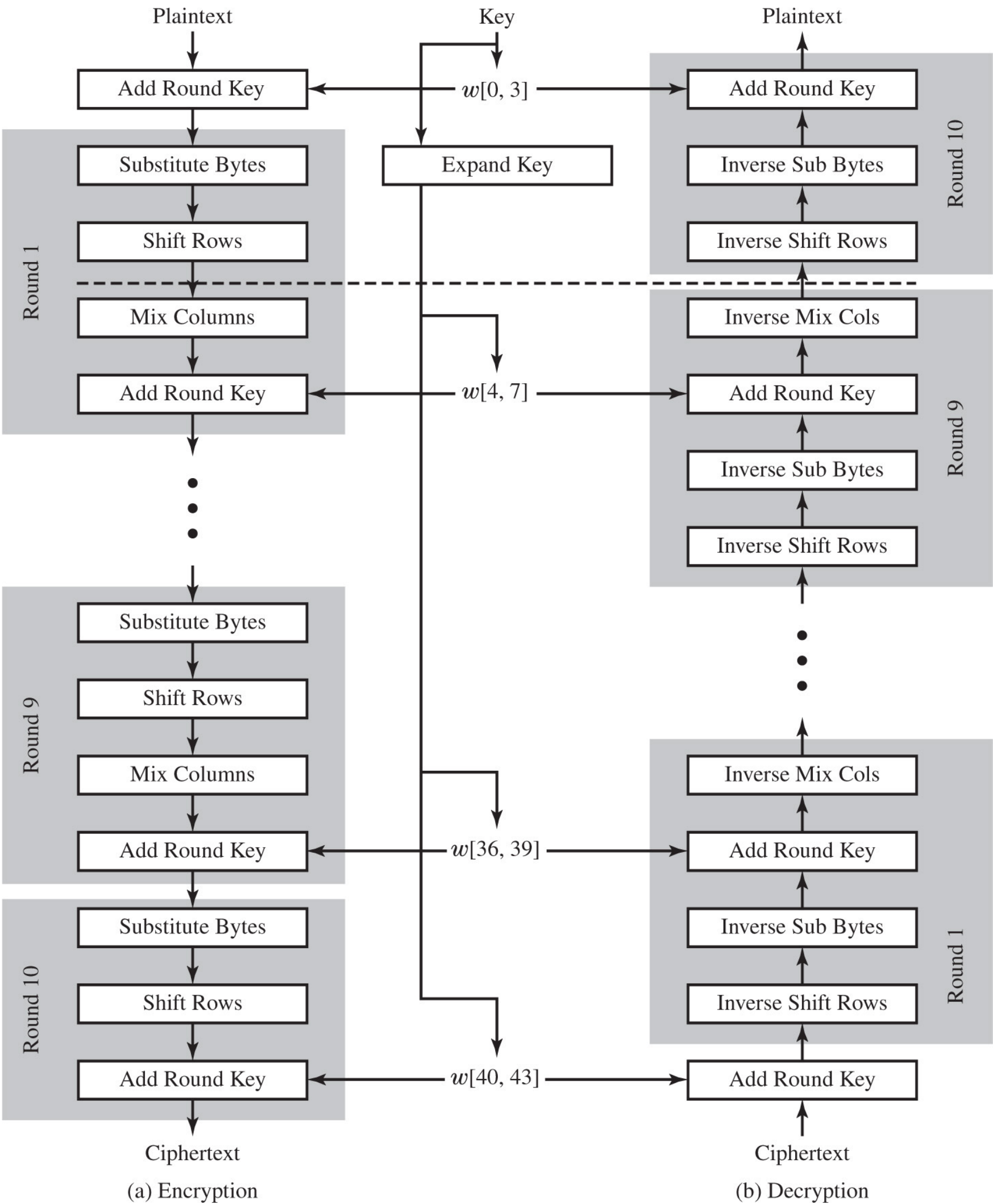
► Example of key reuse



Symmetric Encryption – AES

- ▶ Published in 2001 by NIST as result of an international competition
- ▶ Encrypts data in blocks of 128 bits
- ▶ Keys can be either 128, 192, or 256 bits
- ▶ Consists of 4 operations in 10-14 stages (depending on key length)
 - ▶ **Substitute Bytes:** Uses a table called S-Box to perform a byte-by-byte substitution
 - ▶ **Shift Rows:** A simple permutation that is performed row by row
 - ▶ **Mix Columns:** A substitution that alters each byte in a column as a function of all the bytes in the column
 - ▶ **Add Round Key:** A simple bitwise XOR of the current block with a portion of the expanded key

Symmetric Encryption – AES

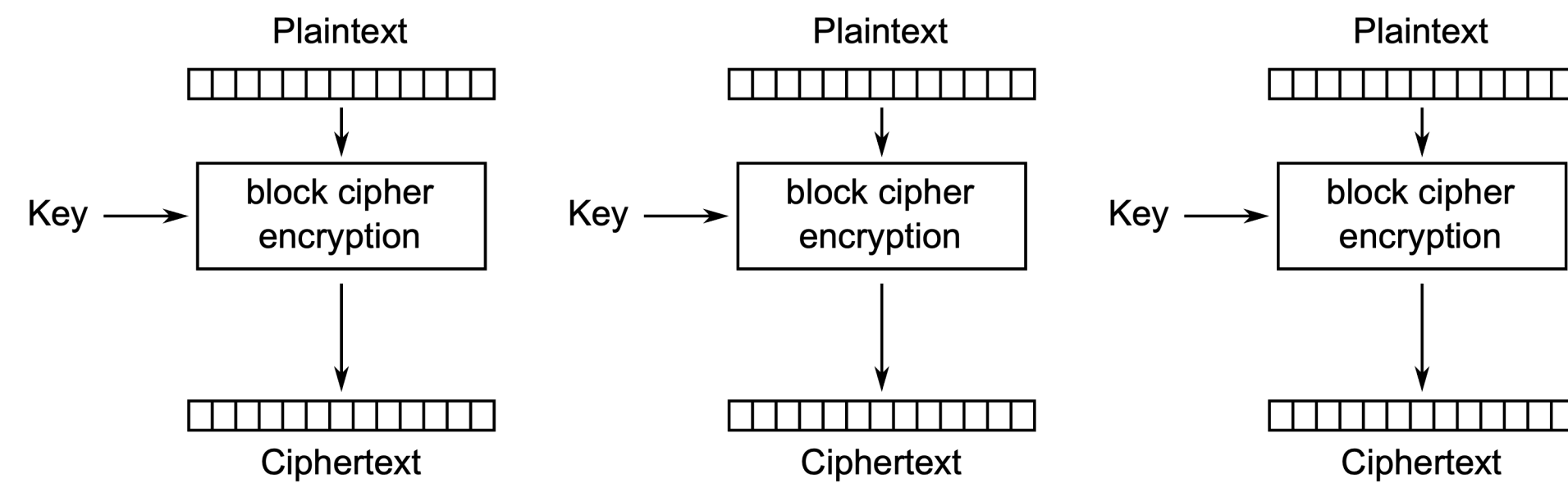


This will not be in the exam

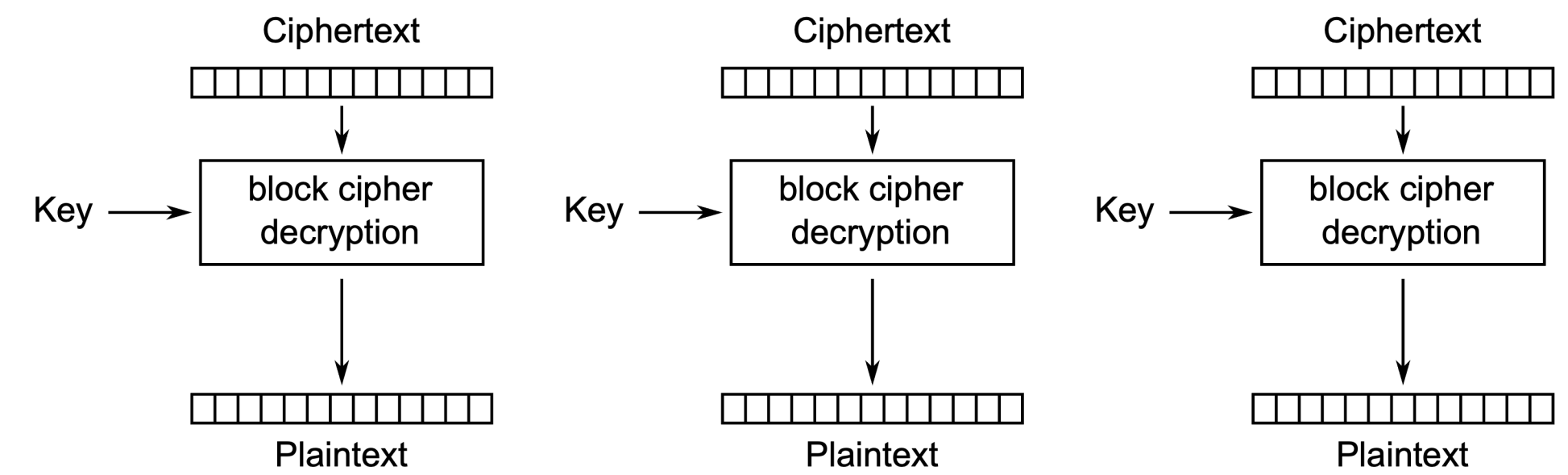
Symmetric Encryption – Modes of Operation

- ▶ Encryption algorithms are divided into two classes
 - ▶ **Stream-ciphers:** Each bit of the data is encrypted and decrypted individually
 - ▶ **Block-ciphers:** The data is divided into blocks of fixed length and each block is encrypted individually
- ▶ Block ciphers require a mode of operation, that handles how the blocks are processed one after the other

Symmetric Encryption – Electronic Codebook



Electronic Codebook (ECB) mode encryption

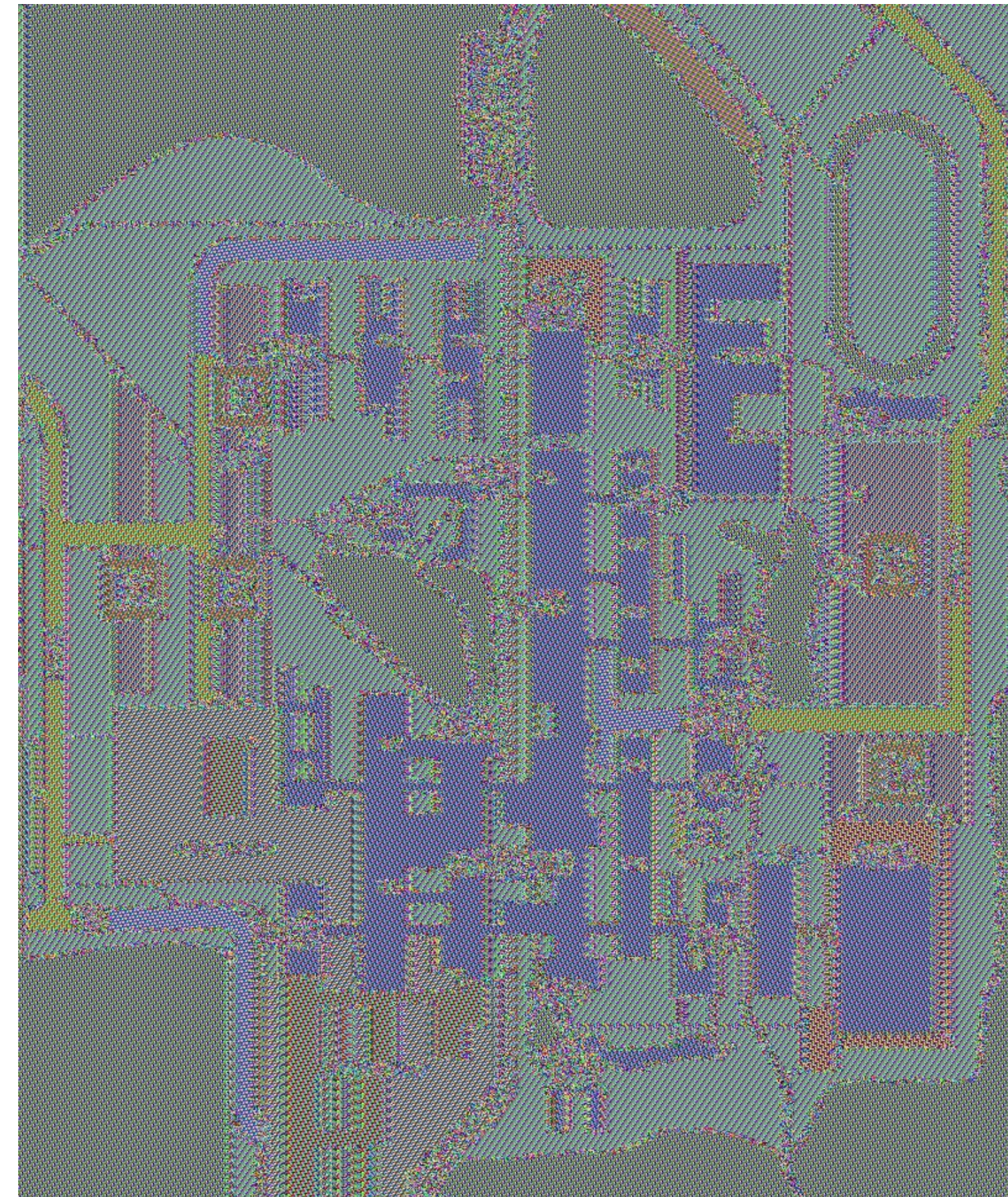


Electronic Codebook (ECB) mode decryption

What is the problem here?

A: Same plaintexts will result in same ciphertexts keeping the structure of the data intact.

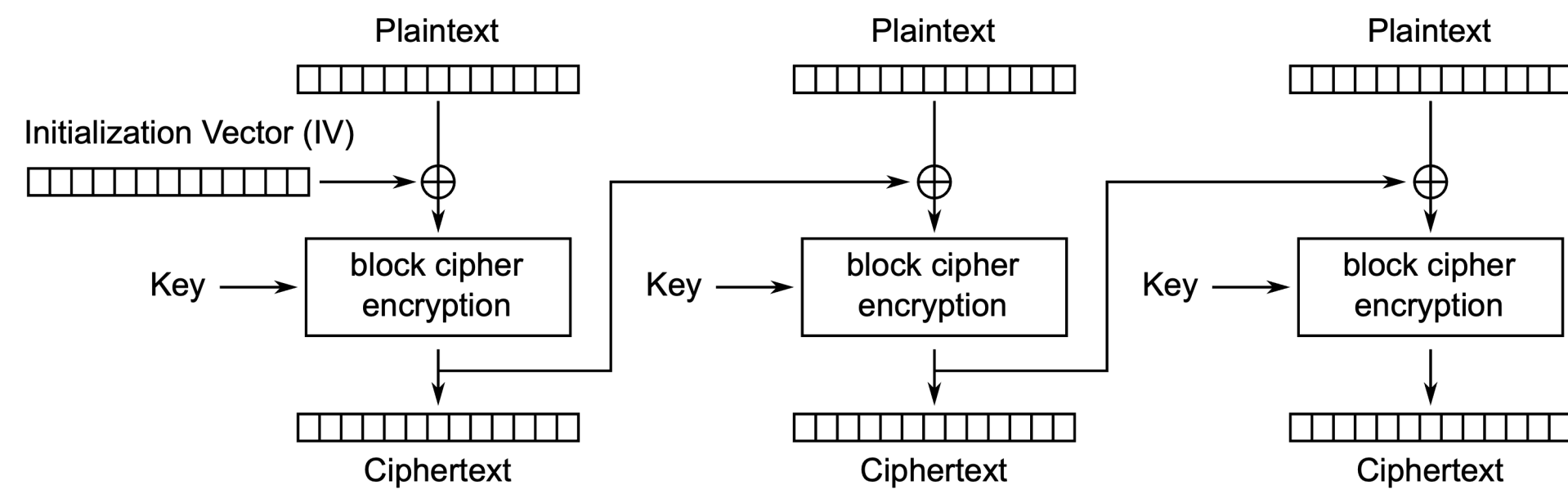
Symmetric Encryption – Electronic Codebook



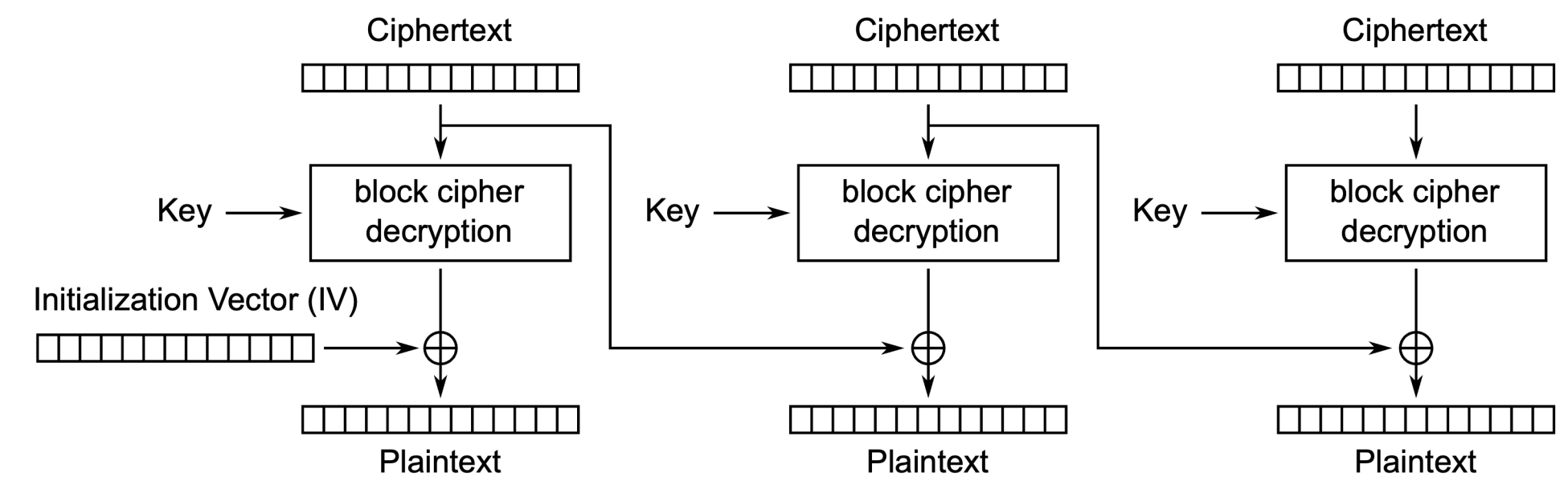
Do not use it!

Symmetric Encryption – Cipher Block Chaining

- Many alternatives exist, e.g., Cipher Block Chaining



Cipher Block Chaining (CBC) mode encryption



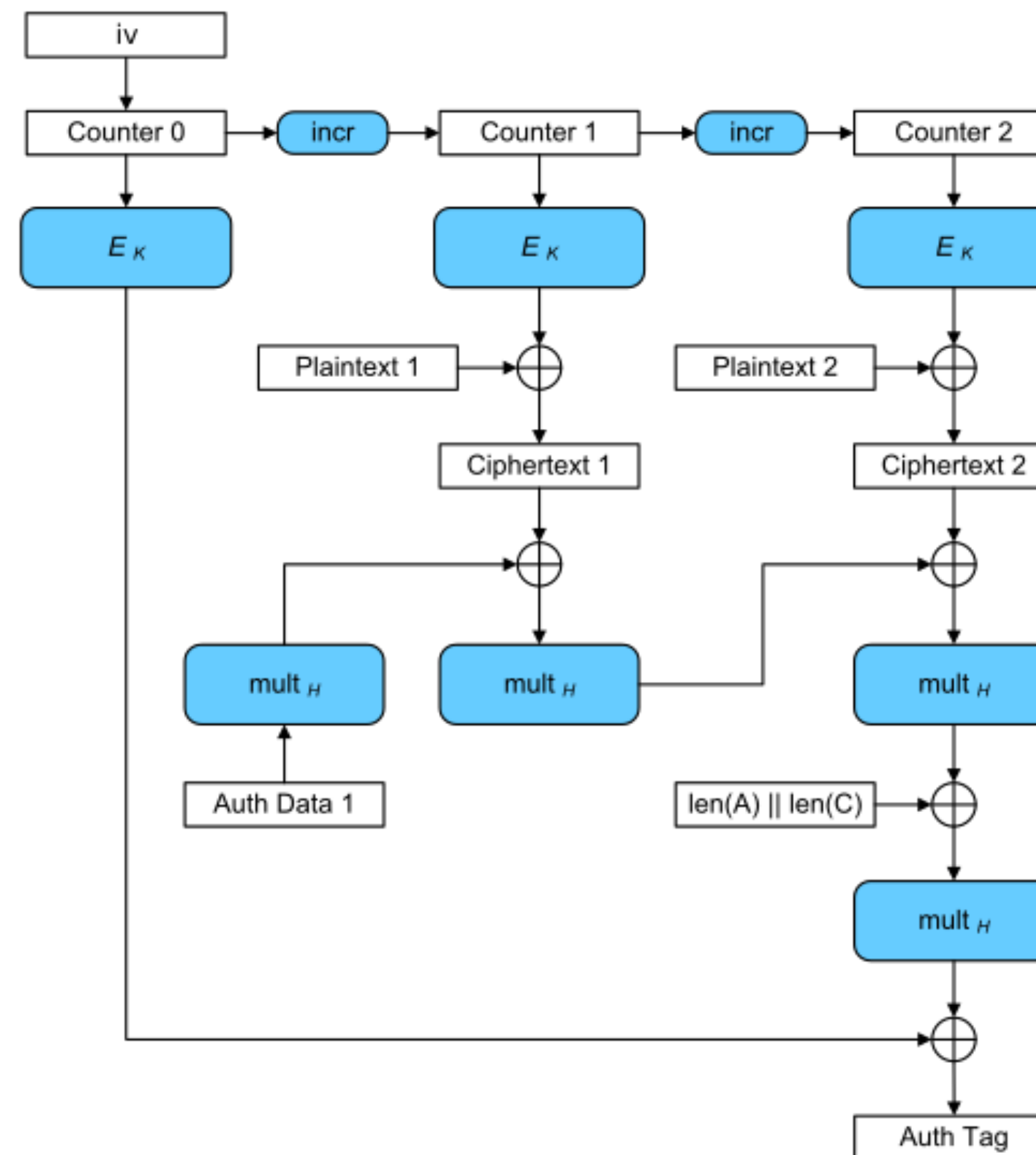
Cipher Block Chaining (CBC) mode decryption



- Much better in terms of security...
- ...but not parallelizable

Symmetric Encryption – Cipher Block Chaining

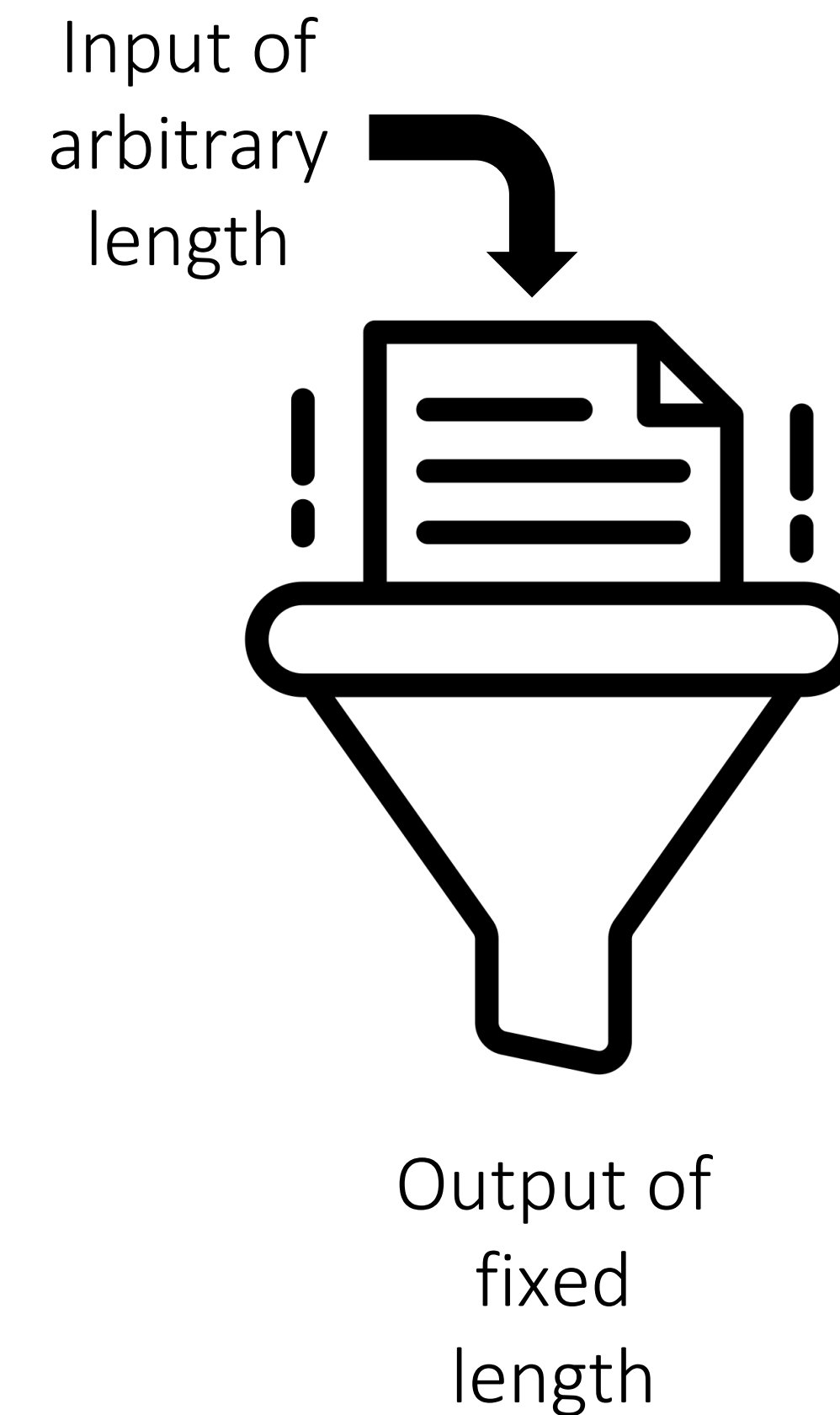
- ▶ Many alternatives exist, e.g., Cipher Block Chaining
- ▶ Currently most advanced one is Galois/Counter Mode



This will not be in the exam

Hash Functions

- ▶ The goal of all hash functions is to derive values of fixed length from inputs of arbitrary length
- ▶ Indicator of integrity
- ▶ Different purposes and properties
 - ▶ Cryptographic Hashes
 - ▶ Hashes for Password Storage



Hash Functions – Cryptographic Hashes

- ▶ **One-way function**

Given output h it is infeasible to find x such that $H(x) = h$

- ▶ **Collision-resistance**

It is infeasible to find any pair (x,y) such that $H(x) = H(y)$

- ▶ **Efficient**

Computation takes little resources

Hash Functions – Cryptographic Hashes

- ▶ Do use (as of 2024)
 - ▶ SHA-3 (preferably)
 - ▶ SHA-2 (SHA-256, SHA-512)

- ▶ Do NOT use
 - ▶ Basically anything not on the list above, but in particular
 - ▶ SHA-1
 - ▶ MD3
 - ▶ MD4
 - ▶ MD5

Hash Functions – Hashes for Password Storage

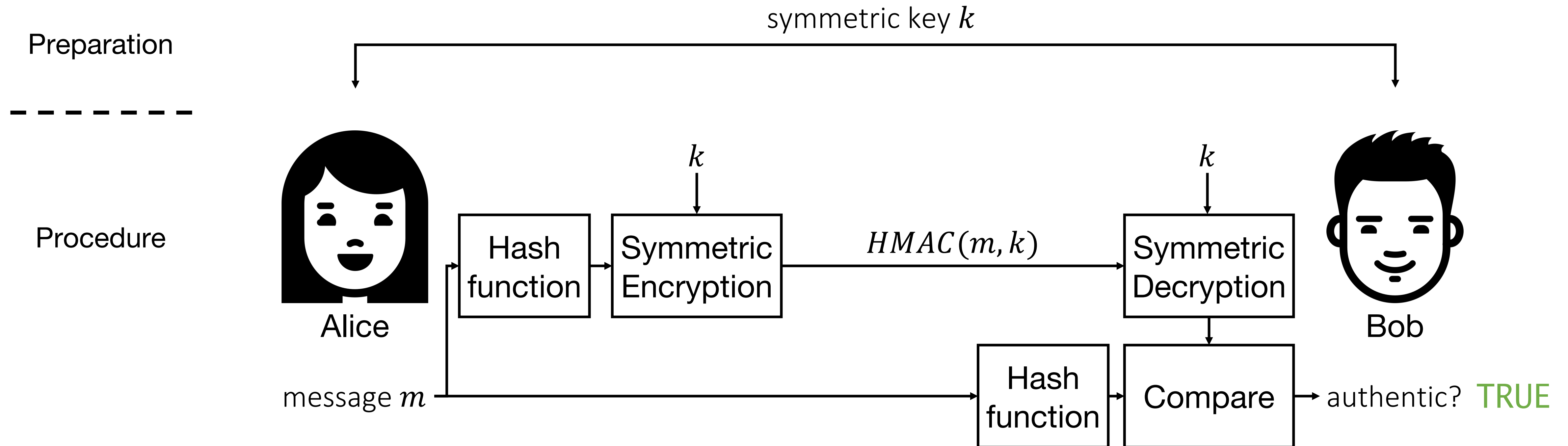
- ▶ Same as cryptographic hashes, except efficient
- ▶ **Computationally difficult**
Guessing attacks should be difficult!
- ▶ Originally no dedicated hashes existed, and cryptographic hashes were used multiple times
 - ▶ PBKDF2 (Password-based key derivation function)
 - ▶ OWASP recommends
 - ▶ 600.000 iterations for HMAC-SHA256
 - ▶ 210.000 iterations for HMAC-SHA512

Hash Functions – Hashes for Password Storage

- ▶ Today specialised hashes for password storage exist
- ▶ Best practice
 - ▶ Argon2 (preferably)
 - ▶ PBKDF2 (if mandated by a certification standard, see configs on previous slide)
- ▶ Not horrible
 - ▶ Bcrypt
 - ▶ Scrypt
- ▶ Do NOT use anything else!
- ▶ There is much more to password storage than just hashes, but we will look at that in a later lecture

Message Authentication Codes

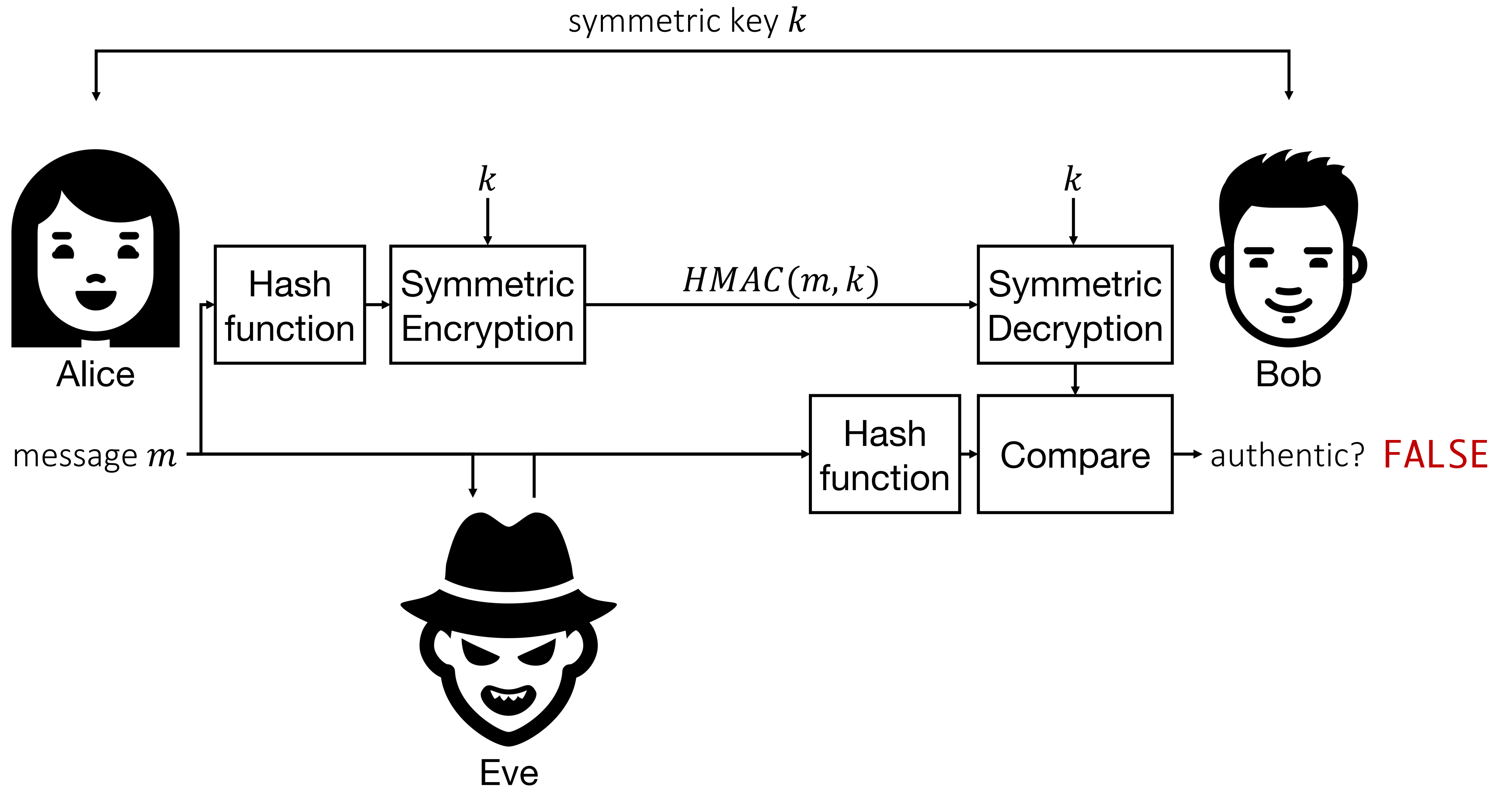
- Provide authenticity of information
- Most prevalent type **HMAC** combines encryption and hash functions



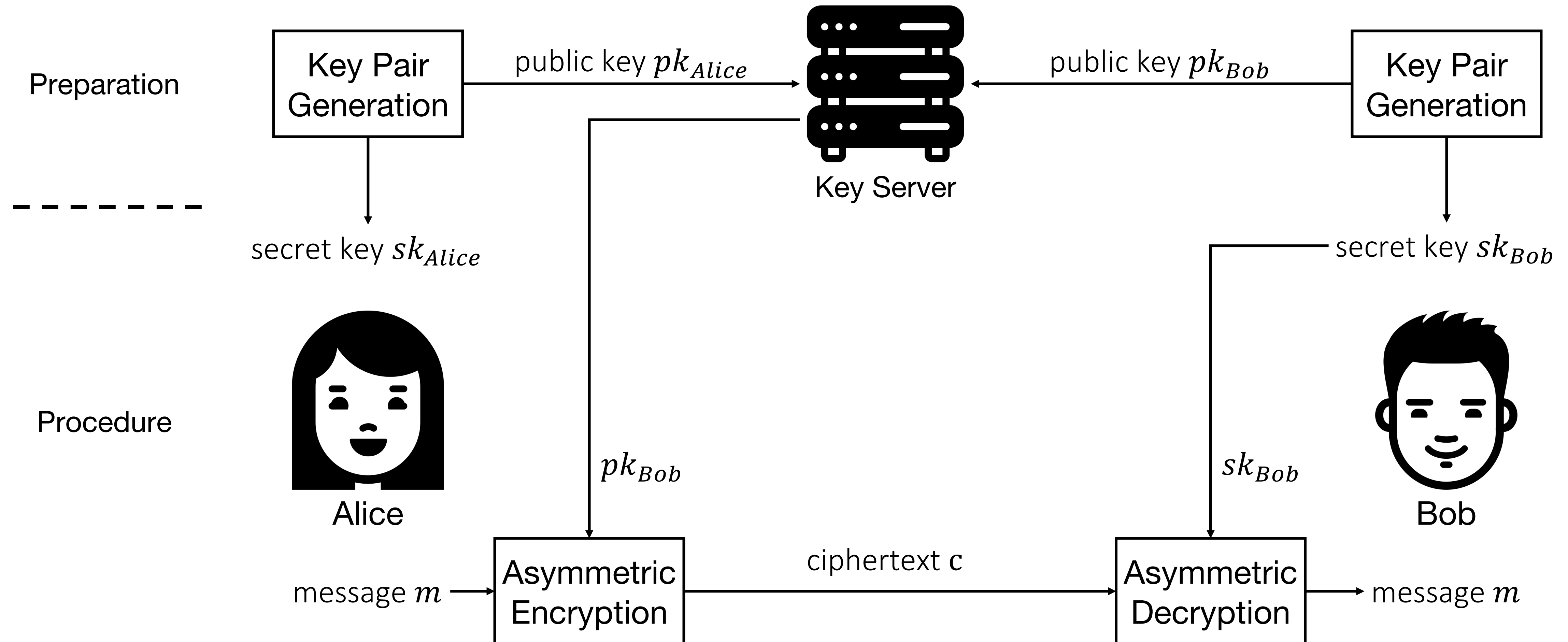
Message Authentication Codes

Preparation

Procedure



Asymmetric Encryption



Asymmetric Encryption – RSA

- ▶ Most widely used asymmetric encryption algorithm
- ▶ Developed in 1977 by Ron Rivest, Adi Shamir, and Len Adleman
- ▶ Security based on the difficulty of factoring large prime numbers
- ▶ Encryption: $c \equiv m^e \pmod{n}$
- ▶ Decryption: $m \equiv c^d \pmod{n} [\equiv (m^e)^d \pmod{n} \equiv m^{ed} \pmod{n}]$
- ▶ e is the public key, d is the private key
- ▶ But how do we get a suitable e and d ?

Asymmetric Encryption – RSA

► Key Generation

1. Select p, q
2. Calculate $n = p * q$
3. Calculate $\phi(n) = (p - 1) * (q - 1)$
4. Select integer e
5. Calculate d

(both p and q prime and $p \neq q$)

($\gcd(\phi(n), e) = 1; 1 < e < \phi(n)$)

($d * e \bmod \phi(n) = 1$)

► Encryption: $c \equiv m^e \bmod n$

($m < n$)

► Decryption: $m \equiv c^d \bmod n [\equiv (m^e)^d \bmod n \equiv m^{ed} \bmod n]$

Asymmetric Encryption – RSA

► Example

1. Select $p = 17, q = 11$
2. Calculate $n = p * q = 17 * 11 = 187$
3. Calculate $\phi(n) = (p - 1) * (q - 1) = 160$
4. Select integer $e = 7$
5. Calculate $d = 23$

(both p and q prime and $p \neq q$)

($\gcd(160, 7) = 1$ and $1 < 7 < 160$)

($23 * 7 \bmod 160 = 1$)

► Encryption: $c \equiv m^e \bmod n$

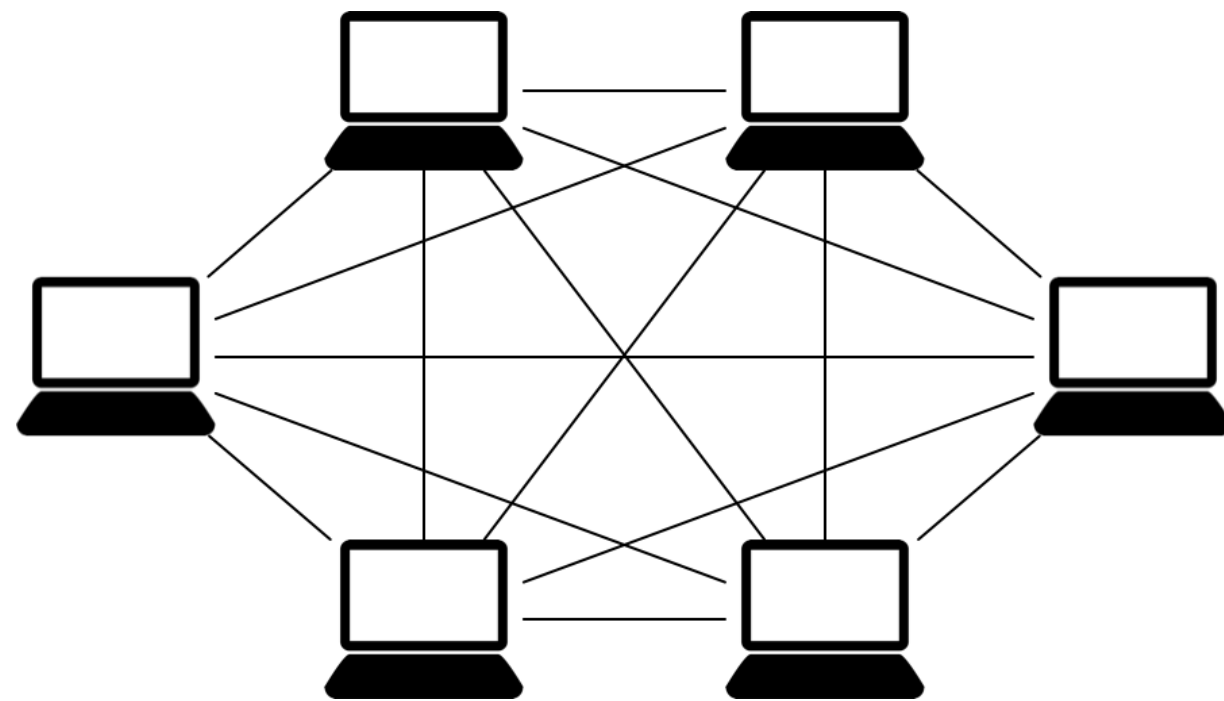
($m < n$)

► Decryption: $m \equiv c^d \bmod n [\equiv (m^e)^d \bmod n \equiv m^{ed} \bmod n]$

Hybrid Encryption

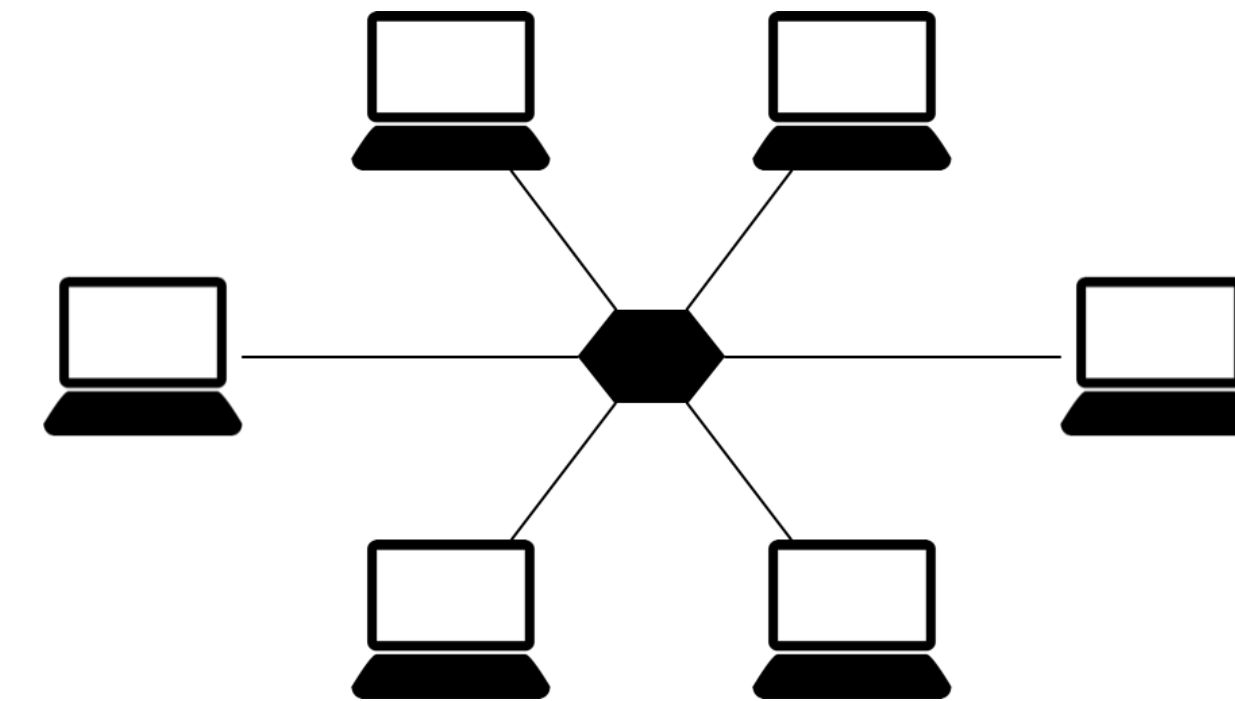
- ▶ Symmetric cryptography is much faster than asymmetric cryptography...
- ▶ ...but key management is more difficult for symmetric keys

Symmetric Cryptography



For n clients: $\binom{n}{2} = \frac{n*(n-1)}{2}$

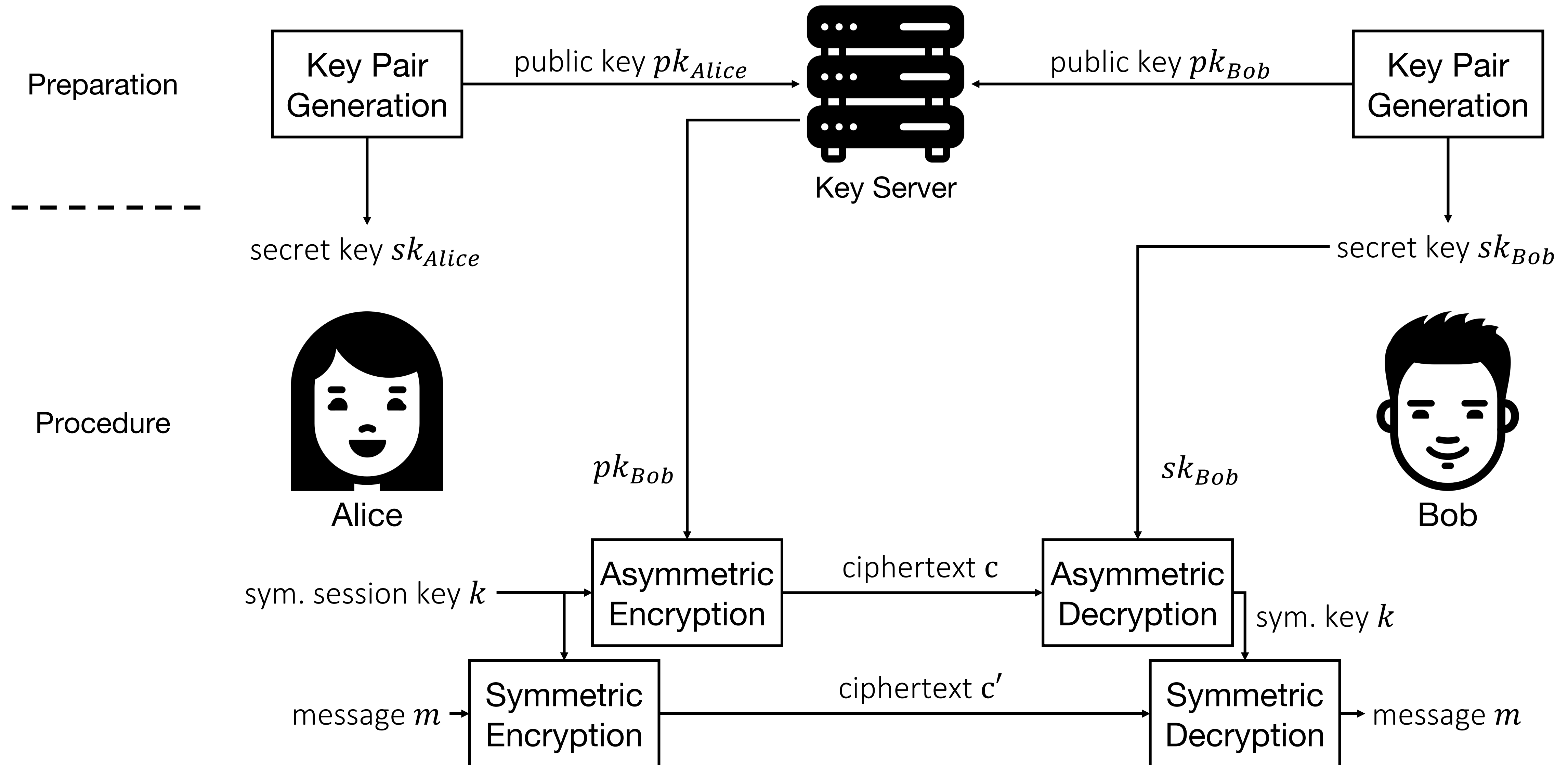
Asymmetric Cryptography



For n clients: n

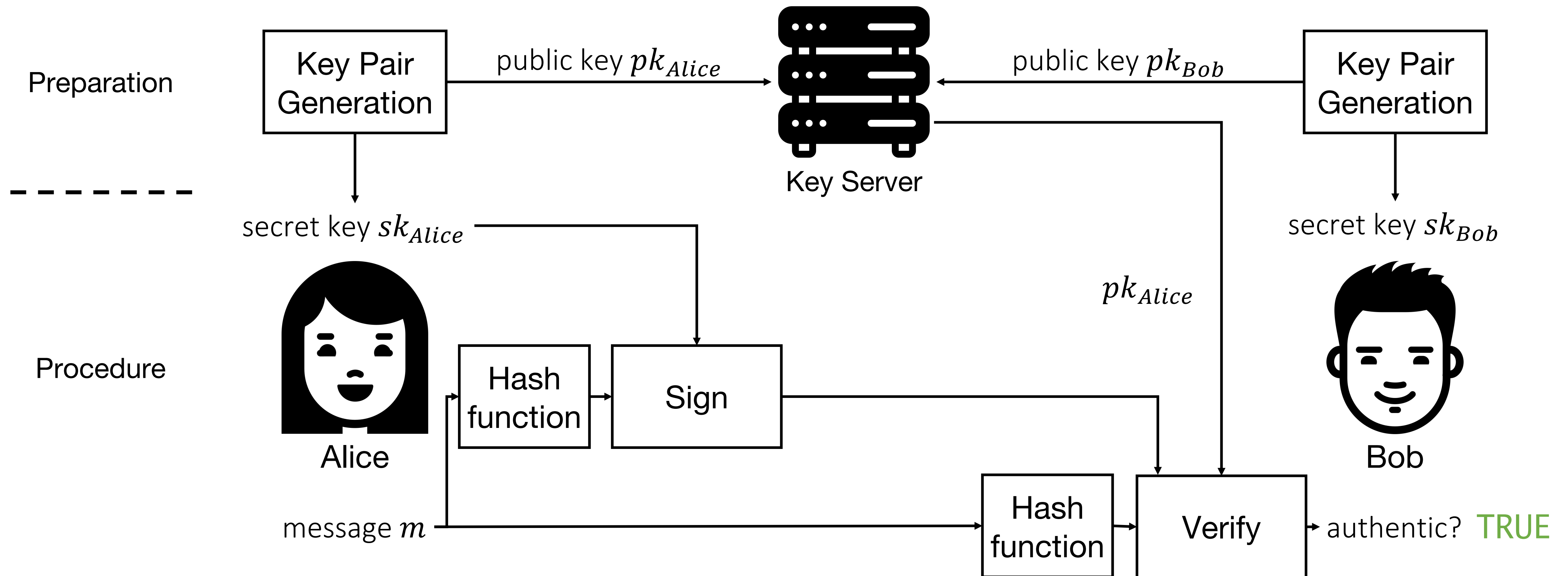
- ▶ Solution combine the two for the best of both worlds
 - ▶ Key management from asymmetric crypto
 - ▶ Data encryption performance from symmetric crypto

Hybrid Encryption

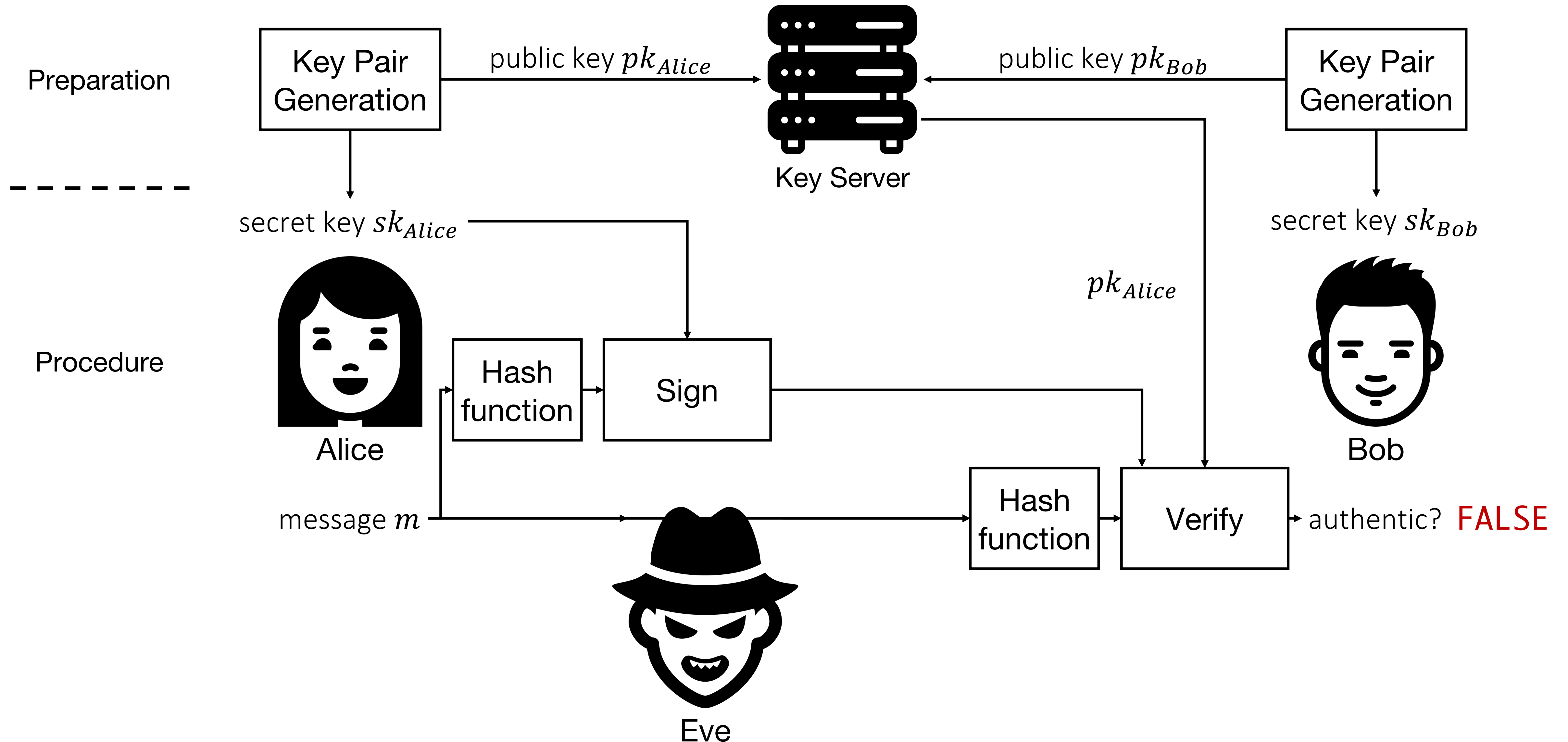


Digital Signatures

- Provide authenticity of data through public key cryptography
- Use RSA or ECDSA (Elliptic Curve Digital Signature Algorithm)



Digital Signatures



Pitfalls in Practice

- ▶ **Common wisdom:** For >99% of us holds
 - ▶ NEVER implement crypto unless you know EXACTLY what you are doing!
 - ▶ ALWAYS use modern but proven algorithms that are not broken!
 - ▶ ALWAYS use proven libraries with good defaults (e.g., NACL)!
- ▶ If you do not follow these rules, you will have a bad time
- ▶ Attacks might lurk where you do not expect it

Pitfalls in Practice – Example 1: Textbook RSA

- ▶ What happens if you implement RSA in exactly the way I explained it to you (textbook version)?
- ▶ Your implementation will be susceptible to fatal attacks, e.g., if you leave out so-called padding
 - ▶ Textbook RSA is deterministic (encrypting the same plaintext twice yields same ciphertext)
 - ▶ Textbook RSA is multiplicatively homomorphic (encrypted values can be multiplicatively modified under encryption)

Pitfalls in Practice – Example 2: Side-channel Attacks

- ▶ Even when the code is secure, hardware might leak your secrets

- ▶ Timing Attacks

- ▶ Operations take different amounts of time depending on the input
- ▶ Measure the runtime for different inputs and deduce the key
- ▶ Make operations fixed-time

- ▶ Power Attacks

- ▶ Operations require different amounts of power
- ▶ Take an oscilloscope to the CPU
- ▶ Measure which operations are performed and deduce the key

Pitfalls in Practice – Example 2: Side-channel Attacks

► And these attacks do not only work on old hardware

RAMBO: Leaking Secrets from Air-Gap Computers by Spelling Covert Radio Signals from Computer RAM

Mordechai Guri^[0000–0003–1806–8858]

Ben-Gurion University of the Negev, Israel
Department of Software and Information Systems Engineering
guri@post.bgu.ac.il
Air-gap research page: <http://www.covertchannels.com>

Abstract. Air-gapped systems are physically separated from external networks, including the Internet. This isolation is achieved by keeping the air-gap computers disconnected from wired or wireless networks, preventing direct or remote communication with other devices or networks. Air-gap measures may be used in sensitive environments where security and isolation are critical to prevent private and confidential information leakage.

In this paper, we present an attack allowing adversaries to leak information from air-gapped computers. We show that malware on a compromised computer can generate radio signals from memory buses (RAM). Using software-generated radio signals, malware can encode sensitive information such as files, images, keylogging, biometric information, and encryption keys. With software-defined radio (SDR) hardware, and a simple off-the-shelf antenna, an attacker can intercept transmitted raw radio signals from a distance. The signals can then be decoded and translated back into binary information. We discuss the design and implementation and present related work and evaluation results. This paper presents fast modification methods to leak data from air-gapped computers at 1000 bits per second. Finally, we propose countermeasures to mitigate this out-of-band air-gap threat.

Keywords: Air-gap · Radio · Electromagnetic · Covert Channels · Exfiltration · RAM · Memory

1 Introduction

Today's regulations, such as GDPR (General Data Protection Regulation), outline strict rules and principles for how organizations should collect, store, and share personal data. It grants individuals certain rights, such as the right to access their data, the right to be forgotten (i.e., to have their data erased), the right to data portability, and more. Organizations that handle personal data must follow certain practices to ensure privacy and security. They need explicit consent from individuals before processing their data. They need to implement

Learning Goals

- ▶ Brief Primer on Cryptography ✓
 - ▶ Symmetric encryption
 - ▶ Hash functions
 - ▶ Message Authentication Codes
 - ▶ Asymmetric encryption
 - ▶ Hybrid encryption
 - ▶ Digital Signatures
 - ▶ Pitfalls when implementing crypto